HEALTH ECONOMICS · PORTFOLIO CASE STUDY

Can AI sepsis alerts pay for themselves?

A cost-effectiveness analysis of AI early-warning systems in hospitals, from the project idea to a working model. It weighs the cost of false alarms and alert fatigue against deaths averted and shorter stays.

Compiled October 25, 2026 · Model outputs under stated assumptions, not observed results

Summary

$88k
net benefit per 1,000 admissions, higher-performing tool at its best threshold

$23k
same measure for a tool that performs like the Epic model did at Michigan

−$2k
net benefit if treatment starts only half an hour earlier (higher-performing tool)

- A widely used sepsis alert caught 33% of cases and was real in only 12% of alerts, about 8 alerts for each real case.
- Raising alert volume trades missed cases for fatigue. Net benefit peaks at a threshold: 16% of patients alerted for the higher-performing tool under moderate fatigue.
- The result depends most on how many hours earlier treatment really starts, then on the mortality effect per hour and the value of a death averted.
- Published mortality drops of 17–18% come from non-randomized implementation studies. A conservative dose-response model implies 2.7% for the better tool.

Open the interactive version with hover tooltips: https://claude.ai/artifact/PYxKXCBfBnBry6×62sP746

1. Where the project started
2. The project plan
3. The story in charts
4. Model and assumptions
5. Limits and what to verify
6. Next steps
Appendix A. Model code
Appendix B. Sources

1. Where the project started

The brief was an AI-in-healthcare cost-effectiveness project. Ideas fell into four groups. Sepsis early-warning systems was chosen because the evidence conflicts, the economics

hinge on false alarms and clinician response, and an independent analysis adds something the one vendor-funded study does not.

Group	Ideas considered
Cost-effectiveness of a specific tool	Diabetic retinopathy screening · AI mammography and lung nodule detection · skin cancer triage · stroke large-vessel detection · **sepsis early warning (chosen)**
Administrative and operational AI	Ambient AI scribes · prior authorization and claims processing · no-show prediction · readmission prediction paired with an intervention
Methodological	Systematic review of AI economic evaluations against CHEERS · a framework for AI-specific costs (retraining, integration, drift) · value-of-information analysis
Equity and adoption	Cost-effectiveness across subgroups when performance differs · rural and low-resource settings · break-even performance analysis

2. The project plan

Research question. Is an AI sepsis early-warning system cost-effective against standard screening once false alarms and alert fatigue are counted, and which alert threshold maximizes net benefit? The alert threshold is treated as a decision variable, because moving along a tool's ROC curve trades missed cases against alert burden, and alert burden changes how often clinicians act.

Design

- Perspective: US hospital and payer; societal as a scenario.
- Population: adults not yet diagnosed with sepsis, per 1,000 admissions.
- Comparators: standard care; an Epic-like tool; a higher-performing tool.
- Short-run decision tree (alert × true sepsis × clinician acts) plus lifetime QALYs for survivors, discounted at 3%.

Alert-fatigue mechanism

- Alerts per real case = 1 / PPV.
- Share of alerts acted on falls as alert volume rises.
- Effective sensitivity = sensitivity × share acted on.
- A higher-sensitivity threshold can lose value if it overwhelms staff.

Analyses

- Base case, threshold and operating-point optimization.
- One-way (tornado) and probabilistic sensitivity analysis.
- Conservative and optimistic effect-size scenarios.
- Planned: value of information, subgroup and equity scenario.

Data and tools

- Performance and effects from the studies cited below.
- Costs and utilization: HCUP, CMS, published sepsis cost studies.
- Alert simulation: MIMIC-IV or eICU (credentialed access).
- Reporting standard: CHEERS 2022. Model in Python; R packages heemod and dampack for a cross-check.

The full plan, including the suggested 8–10 week timeline, is saved in the Healthcare Cost Effectiveness project as *sepsis_ai_cea_project_plan.md*.

3. The story in charts

Each chart answers one question in the argument, in order: how big the problem is, why current tools fall short, what better tools promise, what the threshold choice costs, and how sure the answer is.

The problem: time matters

More than 1.7 million Americans develop sepsis each year and at least 350,000 die, according to CDC figures cited by a patient-advocacy fact sheet, which also puts hospital spending near $38 billion a year. In a New York State study of mandated sepsis care, each hour of delay raised the odds of in-hospital death by a factor of 1.04.

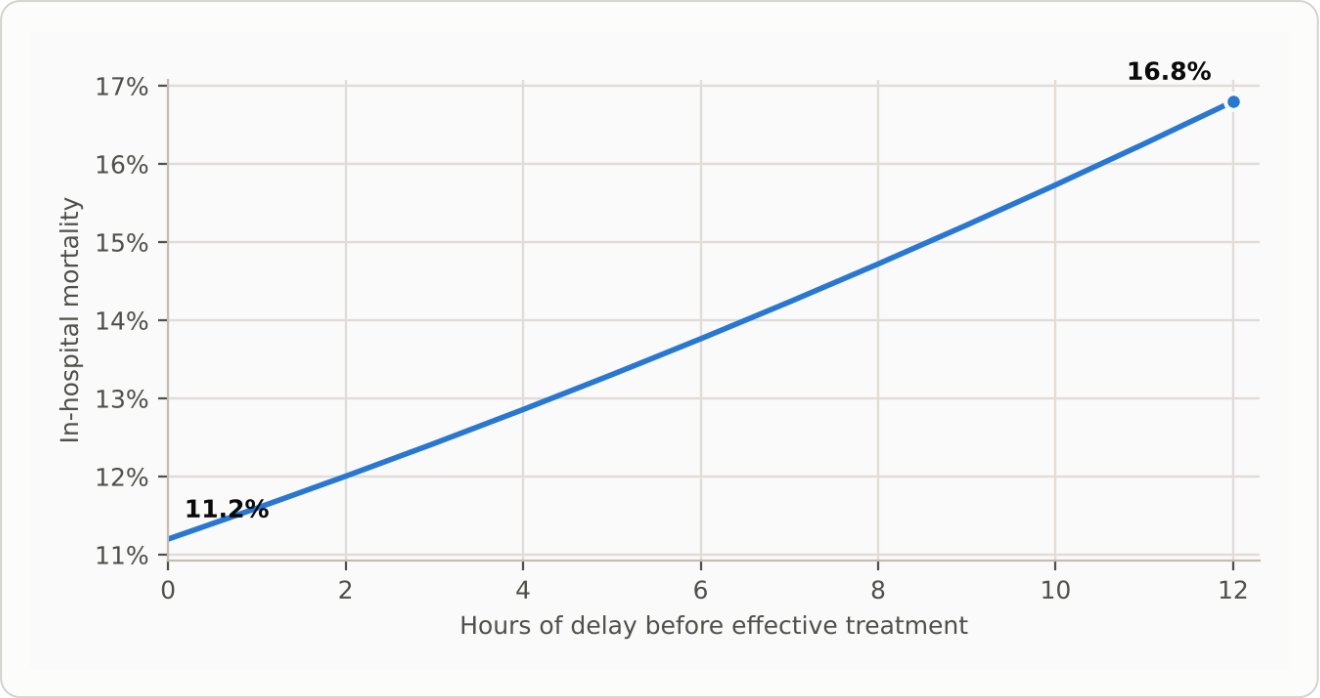


Figure 1. In-hospital mortality for a patient with 11.2% baseline risk if the per-hour odds ratio of 1.04 compounds. Derived from the study estimate, not observed hour by hour.

Today's tools: most cases missed, most alerts false

The external validation of the Epic Sepsis Model covered 38,455 hospitalizations at the University of Michigan. The alert fired on 18% of all patients, caught 33% of sepsis cases and was real in 12% of alerts.

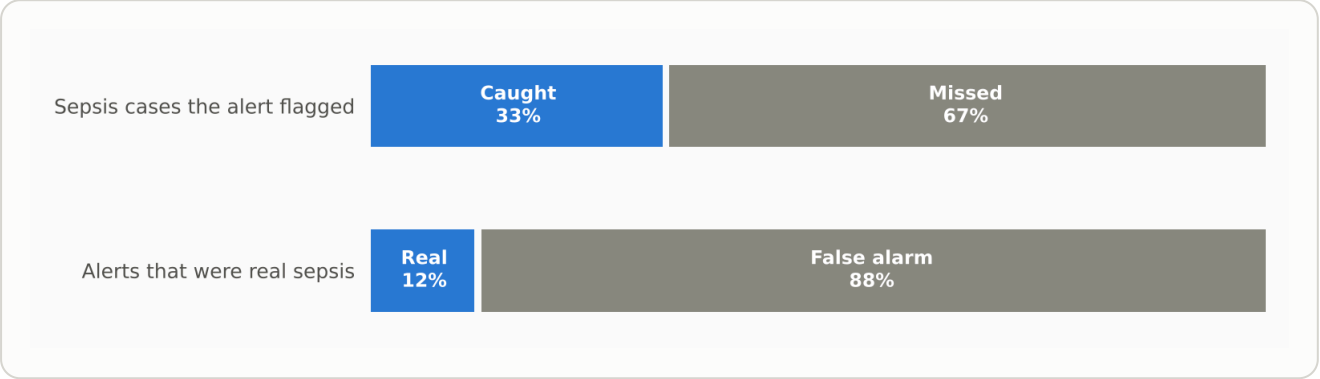


Figure 2. Share of sepsis cases flagged, and share of alerts that were real. Source: Wong et al., JAMA Internal Medicine 2021.

The promise: large reported gains, weaker evidence

TREWS at Johns Hopkins reported an 18% fall in sepsis mortality and COMPOSER at UC San Diego a 17% relative fall. Both are real-world implementation studies, not randomized trials. A conservative model built on the per-hour odds ratio implies a far smaller effect.

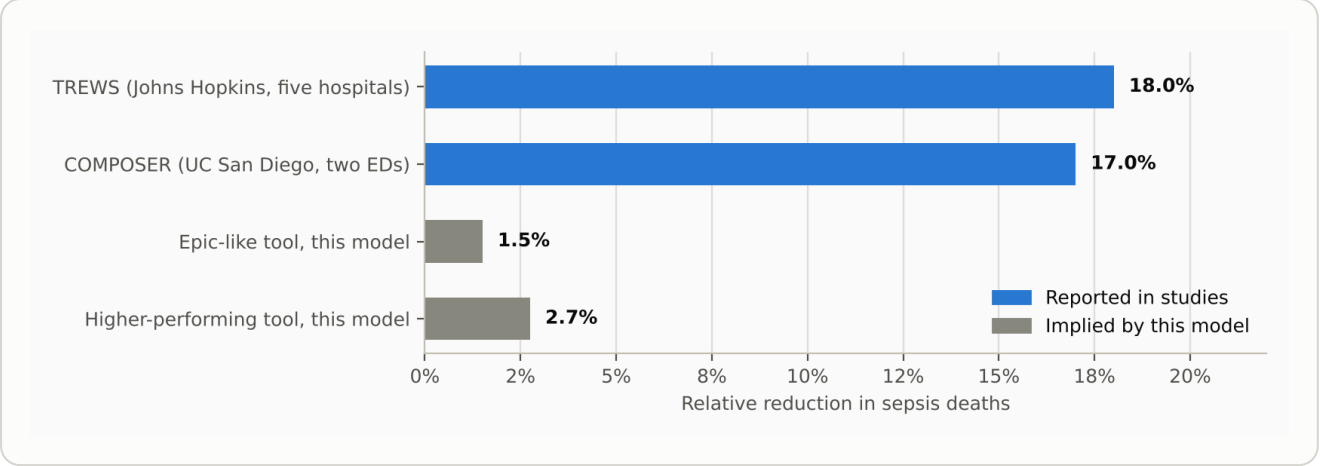


Figure 3. Relative reduction in sepsis deaths, reported and modeled. The modeled bars are each tool at its best threshold under moderate fatigue and the conservative dose-response effect.

The trade-off: what gets flagged is not what gets acted on

Lowering the threshold flags more sepsis but fires on more patients. When the share of alerts clinicians act on falls with volume, the cases actually acted on peak and then decline.

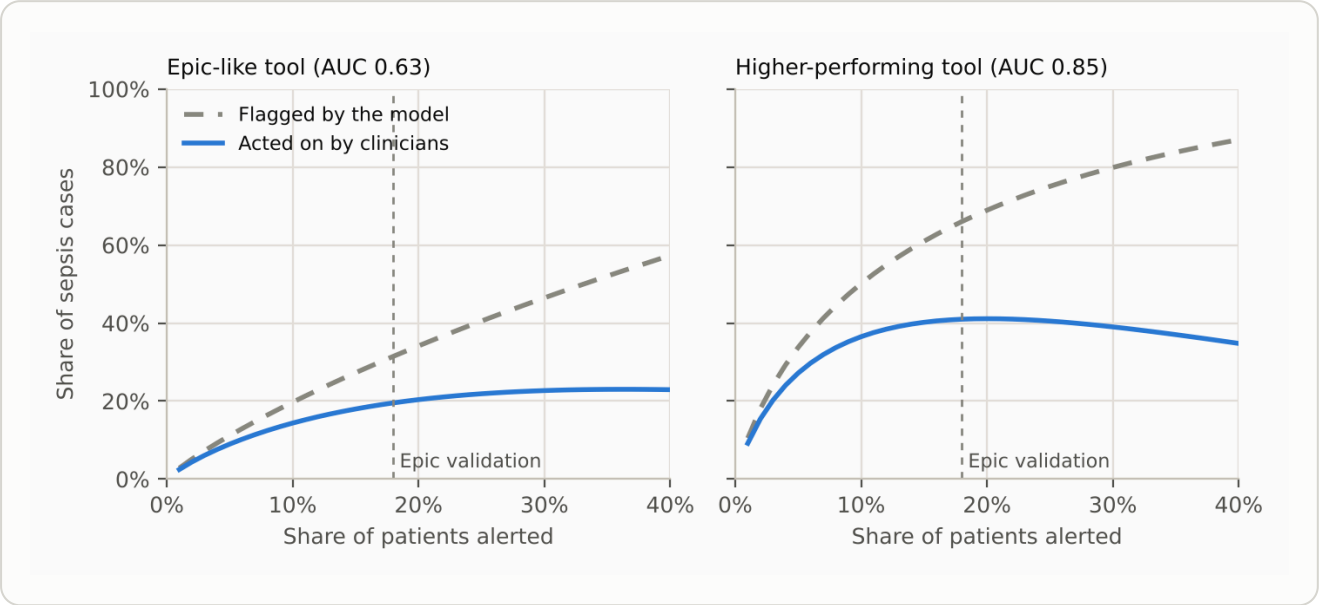


Figure 4. Dashed: share of cases the model flags. Solid: share clinicians act on, under moderate fatigue. The fatigue curve is an assumption.

The value: one best threshold per tool

Net monetary benefit against standard care at $100,000 per QALY. The higher-performing tool peaks at 16% of patients alerted (3.9 alerts per real case). Severe fatigue pulls the best threshold lower and shrinks the benefit for both tools.

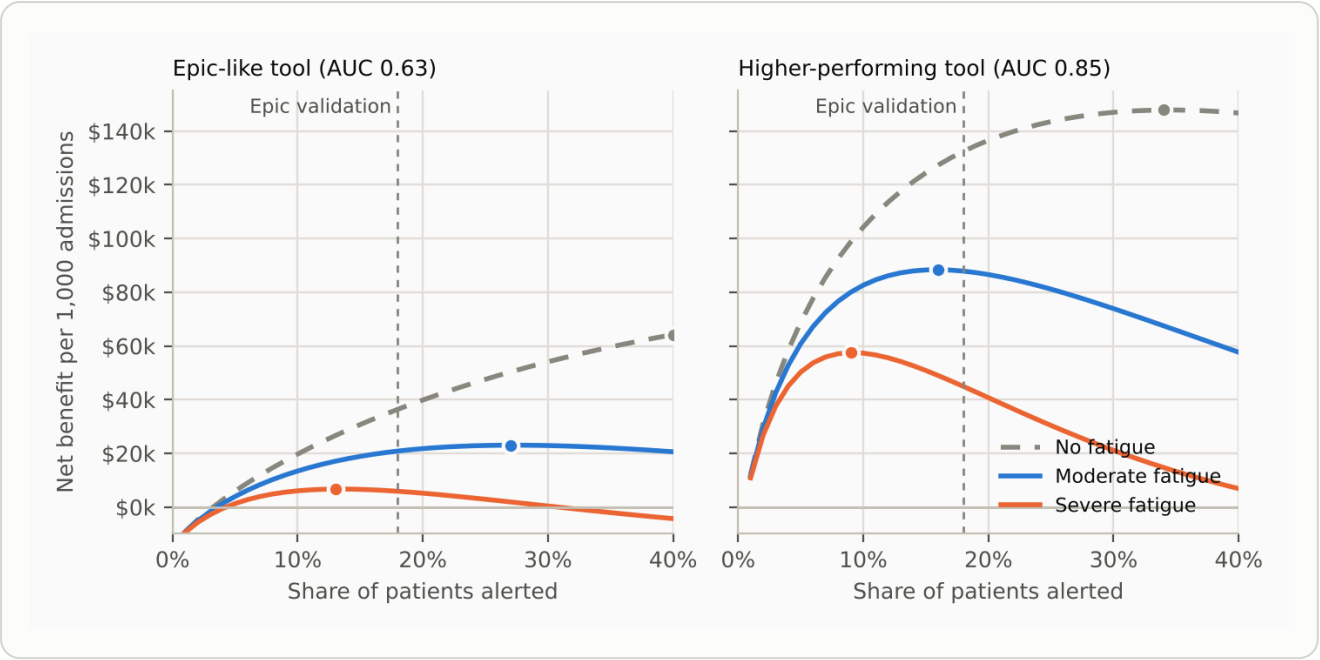


Figure 5. Net benefit per 1,000 admissions. Dots mark each curve's best threshold. The vertical line is the alert rate seen in the Epic validation. Both panels share one scale.

How sure: the probability of being cost-effective

Across probabilistic draws over the ranges in the assumptions table, the higher-performing tool had positive net benefit in 98% of draws at $100,000 per QALY and the Epic-like tool in 75%. Applying COMPOSER's reported relative reduction to every acted-on case (the optimistic effect) lifts both toward certainty.

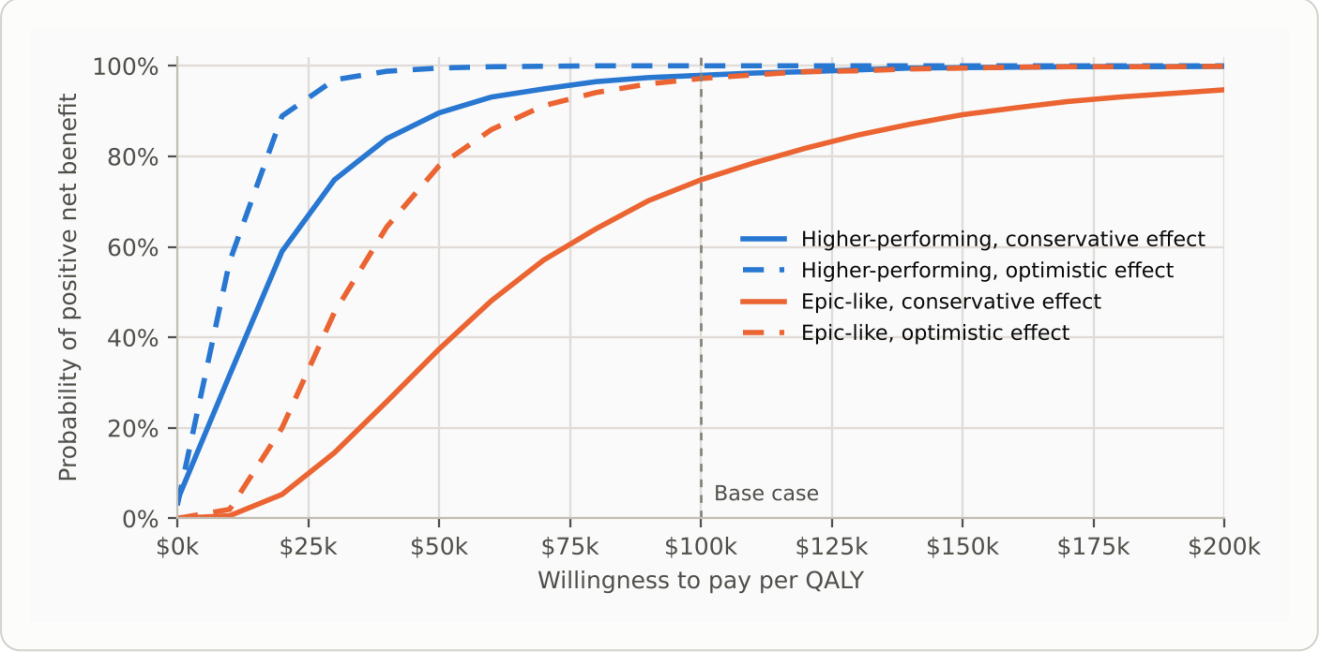


Figure 6. Share of simulated draws with positive net benefit by willingness to pay per QALY.

What moves the answer most

For the higher-performing tool, how many hours earlier treatment starts matters more than anything else. At half an hour the net benefit is −$2k. The effect of an hour of delay on survival and the value of a death averted come next.

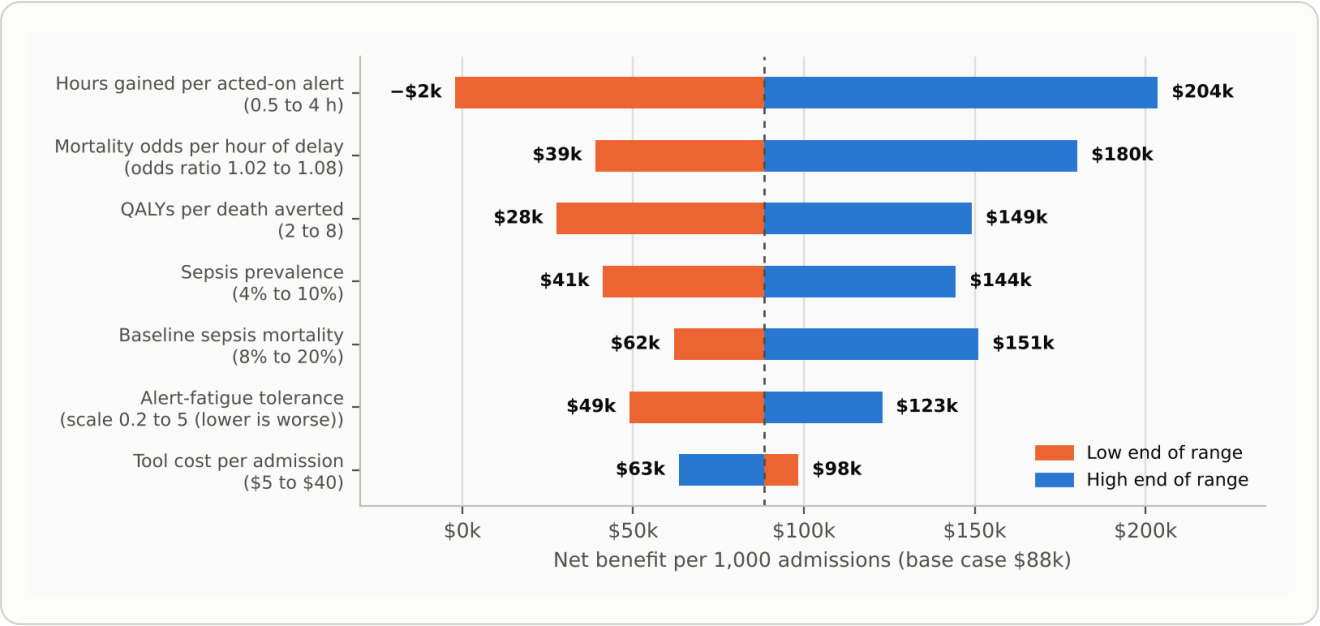


Figure 7. Net benefit at the low and high end of each input, others at base values. Base case $88k per 1,000 admissions.

4. Model and assumptions

The model runs per 1,000 admissions. Each tool is a binormal ROC curve fitted to its AUC. For a chosen share of patients alerted it gives sensitivity and PPV, then applies the alert-fatigue curve, then converts deaths averted into QALYs and adds costs. Calibration check: the Epic-like curve at an 18% alert rate gives 31% sensitivity, 83% specificity and 11.5% PPV, against 33%, 83% and 12% observed in Michigan.

Parameter	Value	Basis	Note
Sepsis prevalence	6.6% of admissions	Derived	From Wong et al.: PPV 12% x alert rate 18% / sensitivity 33%.
Baseline sepsis mortality	11.2%	Derived	COMPOSER: 1.9 pp absolute reduction = 17% relative.
Epic-like tool	AUC 0.63	Sourced	Wong et al. External validation; binormal ROC fitted to the AUC. At 18% alerts it gives 31% sensitivity, 83% specificity, 11.5% PPV.

Parameter	Value	Basis	Note
Higher-performing tool	AUC 0.85	Assumption	A scenario, not a named product.
Mortality effect of earlier treatment	Odds ratio 1.04 per hour	Sourced	Seymour et al. 2017, conservative base case. Range 1.02 to 1.08.
Hours gained per acted-on alert	2 hours (0.5 to 4)	Sourced	TREWS reported treatment about two hours earlier.
Share of alerts acted on, no fatigue	89%	Sourced	TREWS: 89% of alerts were evaluated.
Alert fatigue	Acted-on share = 89% x exp(-alert rate / L)	Assumption	L = 0.5 moderate, 0.2 severe; no fatigue as the upper bound.
QALYs per death averted	5 (2 to 8)	Assumption	Discounted survivor QALYs; to be sourced.
Tool cost	$15 per admission (5 to 40)	Assumption	Licensing, integration and monitoring, spread across admissions.
Clinician time per alert	$25 per evaluated alert (10 to 60)	Assumption	Nurse and physician minutes.
Work-up per false alarm	$200 (100 to 500)	Assumption	Labs, cultures and some unnecessary antibiotics.
Length-of-stay saving	0.15 day per hour gained, $2,500 per day	Assumption	Scaled to the hours of earlier treatment.
Willingness to pay	$100,000 per QALY	Convention	Net monetary benefit = willingness to pay x QALYs gained - net cost.
Comparator	Standard care, no tool	Method	Costs and QALYs are incremental to standard care.
Uncertainty	5,000 probabilistic draws	Method	Triangular or log-uniform distributions over the ranges above.

Net monetary benefit = willingness to pay × QALYs gained − net cost, per 1,000 admissions. "Derived" values are calculated from published figures. "Assumption" values are placeholders that need sources.

5. Limits and what to verify

Verify before this goes in a public portfolio

- The PubMed and PMC pages blocked automated access, so the TREWS and COMPOSER figures come from secondary summaries. Re-pull them from the primary papers.
- The per-hour odds ratio of 1.04 came from a summary of Seymour et al. and applies to bundle completion. The burden figures come from an advocacy fact sheet citing CDC and AHRQ. Check both against the originals.
- This report calls TREWS a non-randomized implementation study. That was not confirmed from the primary paper.
- Replace placeholder unit costs, QALYs per death averted, and the 0.85 AUC scenario with sourced values.

- Effect sizes come from observational and before-and-after studies. A randomized trial would change the central estimate.
- The alert-fatigue curve is an assumption. Studies report how often alerts were evaluated, not how response falls as volume rises.
- A binormal ROC curve stands in for each tool's real score distribution.
- Harms from unnecessary antibiotics are folded into one work-up cost per false alarm.
- The Epic-like tool stays cost-effective in most draws partly because the placeholder unit costs are low relative to $100,000 per QALY.

6. Next steps

Strengthen the evidence

- Source every placeholder cost and the QALY value.
- Search for newer Epic model versions and randomized trials.
- Estimate alert volume at each threshold from MIMIC-IV.

Extend the model

- Add a clinician-time and burnout view of alert burden.
- Add an equity scenario where performance differs by subgroup.
- Run value-of-information analysis to see whether a trial is worth it.

Appendix A. Model code

Python with NumPy and SciPy. Run it with an output path to write the JSON results behind Figures 3 to 7.

```
"""
Illustrative cost-effectiveness model: AI sepsis early-warning tool vs. standard care.
Per 1,000 hospital admissions. Binormal ROC; alert-fatigue response curve.

SOURCED / DERIVED INPUTS
  prevalence 6.6%     derived from Wong et al. 2021 (PPV 12%, sens 33%, alert rate 18%)
  AUC 0.63            Wong et al. 2021 (Epic Sepsis Model external validation)
  baseline mort 11.2% derived from COMPOSER (1.9 pp absolute = 17% relative)
  OR 1.04 per hour    Seymour et al. NEJM 2017 (bundle completion delay)
  P(act) ceiling 0.89 TREWS: 89% of alerts evaluated
  hours gained 2      TREWS: treatment ~2 h earlier
ASSUMPTIONS (placeholders to be sourced; varied in sensitivity analysis)
  AUC 0.85 for the 'higher-performing' tool, fatigue scale, QALYs per death
  averted, all unit costs.
"""
import json, math, sys
import numpy as np
from scipy.stats import norm
from scipy.optimize import brentq

N = 1000
BASE = dict(
    prev=0.066, m0=0.112, hours=2.0, or_h=1.04, p_max=0.89, L=0.5,
    qaly=5.0, handle=25.0, fp_work=200.0, tool=15.0,
    los_days_per_hour=0.15, day_cost=2500.0, wtp=100000.0, effect="seymour",
    rel_red=0.17,
)
TOOLS = {"Epic-like (AUC 0.63)": 0.63, "Higher-performing (AUC 0.85)": 0.85}


def op_point(auc, alert_rate, prev):
    d = math.sqrt(2) * norm.ppf(auc)
    f = lambda c: prev * (1 - norm.cdf(c - d)) + (1 - prev) * (1 - norm.cdf(c)) - alert_rate
    c = brentq(f, -6, 8)
    sens = 1 - norm.cdf(c - d)
    return sens, 1 - norm.cdf(c)


def evaluate(auc, alert_rate, fatigue=True, **over):
    p = {**BASE, **over}
    sens, fpr = op_point(auc, alert_rate, p["prev"])
    sepsis = p["prev"] * N
    tp = sens * sepsis
    alerts = alert_rate * N
    fp = alerts - tp
    ppv = tp / alerts
    p_act = p["p_max"] * (math.exp(-alert_rate / p["L"]) if fatigue else 1.0)
    atp, afp = tp * p_act, fp * p_act
    m0 = p["m0"]
    if p["effect"] == "seymour":
        odds = m0 / (1 - m0) / (p["or_h"] ** p["hours"])
        m1 = odds / (1 + odds)
        averted_per = m0 - m1
    else:
        averted_per = m0 * p["rel_red"]
    deaths_averted = atp * averted_per
    qalys = deaths_averted * p["qaly"]
    cost_tool = p["tool"] * N
    cost_handle = alerts * p_act * p["handle"]
    cost_fp = afp * p["fp_work"]
    offset = atp * p["los_days_per_hour"] * p["hours"] * p["day_cost"]
    net_cost = cost_tool + cost_handle + cost_fp - offset
    nmb = p["wtp"] * qalys - net_cost
    return dict(alert_rate=alert_rate, sens=sens, spec=1 - fpr, ppv=ppv,
                alerts_per_tp=alerts / tp, p_act=p_act, eff_sens=sens * p_act,
```

```
                deaths_averted=deaths_averted, qalys=qalys, net_cost=net_cost,
                nmb=nmb, cost_tool=cost_tool, cost_handle=cost_handle,
                cost_fp=cost_fp, offset=offset,
                icer=(net_cost / qalys if qalys > 0 else None))


GRID = np.round(np.arange(0.01, 0.401, 0.01), 3)


def sweep(auc, fatigue=True, **over):
    return [evaluate(auc, a, fatigue, **over) for a in GRID]


def best(rows):
    return max(rows, key=lambda r: r["nmb"])


def tri(rng, lo, mode, hi, n):
    return rng.triangular(lo, mode, hi, n)


def psa(auc, alert_rate, n=5000, seed=7, effect="seymour"):
    rng = np.random.default_rng(seed)
    draws = dict(
        hours=tri(rng, 0.5, 2.0, 4.0, n), or_h=tri(rng, 1.02, 1.04, 1.08, n),
        L=np.exp(rng.uniform(math.log(0.2), math.log(5), n)),
        qaly=tri(rng, 2, 5, 8, n), handle=tri(rng, 10, 25, 60, n),
        fp_work=tri(rng, 100, 200, 500, n), tool=tri(rng, 5, 15, 40, n),
        p_max=tri(rng, 0.7, 0.89, 0.95, n), m0=tri(rng, 0.08, 0.112, 0.20, n),
        rel_red=tri(rng, 0.05, 0.17, 0.25, n),
    )
    out = []
    for i in range(n):
        o = {k: float(v[i]) for k, v in draws.items()}
        r = evaluate(auc, alert_rate, True, effect=effect, **o)
        out.append((r["qalys"], r["net_cost"]))
    return np.array(out)


def ceac(arr, wtps):
    q, c = arr[:, 0], arr[:, 1]
    return [float(np.mean(w * q - c > 0)) for w in wtps]


def tornado(auc, alert_rate):
    ranges = dict(
        hours=(0.5, 4.0), or_h=(1.02, 1.08), L=(0.2, 5.0), qaly=(2, 8),
        handle=(10, 60), fp_work=(100, 500), tool=(5, 40), p_max=(0.7, 0.95),
        m0=(0.08, 0.20), prev=(0.04, 0.10),
    )
    base = evaluate(auc, alert_rate)["nmb"]
    rows = []
    for k, (lo, hi) in ranges.items():
        a = evaluate(auc, alert_rate, **{k: lo})["nmb"]
        b = evaluate(auc, alert_rate, **{k: hi})["nmb"]
        rows.append(dict(param=k, lo=lo, hi=hi, nmb_lo=a, nmb_hi=b, swing=abs(b - a)))
    rows.sort(key=lambda r: -r["swing"])
    return base, rows


if __name__ == "__main__":
    out = {"base": BASE, "grid": GRID.tolist(), "tools": {}}
    wtps = list(range(0, 200001, 10000))
    out["wtps"] = wtps
    for name, auc in TOOLS.items():
        t = {}
        t["fatigue"] = sweep(auc, True)
        t["nofatigue"] = sweep(auc, False)
        t["fatigue_optimistic"] = sweep(auc, True, effect="observed")
        t["fatigue_severe"] = sweep(auc, True, L=0.2)
        t["best_severe"] = best(t["fatigue_severe"])
        t["at18"] = {
            "none": evaluate(auc, 0.18, False),
```

```
            "moderate": evaluate(auc, 0.18, True),
            "severe": evaluate(auc, 0.18, True, L=0.2),
        }
        t["best_fatigue"] = best(t["fatigue"])
        t["best_nofatigue"] = best(t["nofatigue"])
        t["best_optimistic"] = best(t["fatigue_optimistic"])
        ar = t["best_fatigue"]["alert_rate"]
        arr = psa(auc, ar)
        t["psa_alert_rate"] = ar
        t["ceac"] = ceac(arr, wtps)
        t["psa_p_ce_100k"] = float(np.mean(100000 * arr[:, 0] - arr[:, 1] > 0))
        t["psa_mean_qaly"] = float(arr[:, 0].mean())
        t["psa_mean_cost"] = float(arr[:, 1].mean())
        arr_o = psa(auc, t["best_optimistic"]["alert_rate"], effect="observed")
        t["ceac_optimistic"] = ceac(arr_o, wtps)
        t["psa_p_ce_100k_optimistic"] = float(np.mean(100000 * arr_o[:, 0] - arr_o[:, 1] > 0))
        base_nmb, tor = tornado(auc, max(ar, 0.02))
        t["tornado_base"] = base_nmb
        t["tornado"] = tor
        out["tools"][name] = t
    # Wong calibration check (Epic-like tool at 18% alert rate, no change in operating point)
    out["calibration"] = {"epic_at_18pct": evaluate(0.63, 0.18, True)}
    json.dump(out, open(sys.argv[1], "w"), indent=1)
    for name, t in out["tools"].items():
        b = t["best_fatigue"]
        print(name)
        print("  best (fatigue) alert rate %.2f sens %.2f ppv %.3f alerts/TP %.1f NMB/1000 $%.0f deaths
averted/1000 %.2f net cost $%.0f ICER %s" % (
            b["alert_rate"], b["sens"], b["ppv"], b["alerts_per_tp"], b["nmb"], b["deaths_averted"], b["net_cost"],
b["icer"]))
        b = t["best_nofatigue"]
        print("  best (no fatigue) alert rate %.2f NMB $%.0f" % (b["alert_rate"], b["nmb"]))
        b = t["best_optimistic"]
        print("  best (optimistic effect) alert rate %.2f NMB $%.0f deaths averted %.2f" % (b["alert_rate"],
b["nmb"], b["deaths_averted"]))
        b = t["best_severe"]
        print("  best (severe fatigue L=0.2) alert rate %.2f NMB $%.0f" % (b["alert_rate"], b["nmb"]))
        print("  at 18%% alert rate NMB/1000: none %.0f moderate %.0f severe %.0f; p_act mod %.2f sev %.2f" % (
            t["at18"]["none"]["nmb"], t["at18"]["moderate"]["nmb"], t["at18"]["severe"]["nmb"],
            t["at18"]["moderate"]["p_act"], t["at18"]["severe"]["p_act"]))
        print("  PSA P(NMB>0 @100k): conservative %.2f  optimistic %.2f" % (t["psa_p_ce_100k"],
t["psa_p_ce_100k_optimistic"]))
        print("  tornado:", [(r["param"], round(r["swing"])) for r in t["tornado"][:5]])
    c = out["calibration"]["epic_at_18pct"]
    print("Calibration Epic-like @18%%: sens %.2f spec %.2f ppv %.3f (Wong: 0.33/0.83/0.12)" % (c["sens"],
c["spec"], c["ppv"]))
```

Appendix B. Sources

1. Wong et al., External validation of a widely implemented proprietary sepsis prediction model, JAMA Internal Medicine 2021
2. Scientific American coverage of the TREWS study (Nature Medicine and npj Digital Medicine, 2022)
3. Impact of a deep learning sepsis prediction model on quality of care and survival (COMPOSER), npj Digital Medicine 2024
4. Seymour et al., Time to treatment and mortality during mandated emergency care for sepsis, NEJM 2017 (summary)
5. End Sepsis fact sheet 2023 (burden figures citing CDC and AHRQ HCUP)
6. The potential cost and cost-effectiveness impact of a machine learning algorithm for early detection of sepsis in ICUs in Sweden, JHEOR